KATEDRA INFORMATIKY
UNIVERZITA PALACKÉHO V OLOMOUCI
www.inf.upol.cz

Michal Krupka
krupka.inf.upol.cz
michal.krupka@upol.cz
17. listopadu 12, 771 46 Olomouc

Paradigmata programování 1 ⋄ poznámky k přednášce

12. Interpret

verze z 11. prosince 2024

1 Implementace vyhodnocovacího procesu

Začátkem semestru jsme se seznamovali s vyhodnocovacím procesem jazyka Lisp. To je postup, jehož účelem je k danému výrazu zjišťovat jeho hodnotu (*vyhodnocovat ho*). Funkce napsaná v Lispu pracuje tak, že postupně vyhodnocuje výrazy, takže popsat, jak fungují programy v Lispu, obnáší popsat vyhodnocovací proces.

Vyhodnocovací proces pracuje se **symbolickými výrazy** (atomy a neprázdnými seznamy), se kterými jsme se později naučili pracovat: vytvářet je a zkoumat jejich obsah.

V principu by pro nás mělo být možné napodobit programátory *PP Polyglotu* a implementovat samotný vyhodnocovací proces. Je to ze dvou shora uvedených důvodů:

1. zdrojový kód je datová struktura, se kterou umíme pracovat,
2. vyhodnocovací proces je přesně popsaný.

K implementování vyhodnocovacího procesu bude stačit napsat jednu funkci, která jako argument přijme výraz a jako výsledek vrátí jeho hodnotu.

V této přednášce takovou funkci napíšeme. Bude se jmenovat `ss-eval` („`ss`“ jako *Simple Scheme*, viz dále) a bude implementovat vyhodnocovací proces podstatně zjednodušené varianty našeho jazyka Lisp.

2 Jazyk Scheme

Naše zjednodušená varianta Lispu bude vycházet z jazyka **Scheme**, který sám je jednoduchou verzí Lispu. Z tohoto jazyka také vychází známý jazyk *Racket*.

Jazyk Scheme používá jednodušší vyhodnocovací proces než Lisp, což nám zjednoduší implementaci. Rozdíl spočívá v tom, že ve Scheme **symboly nemají funkční vazby**. Pokud chceme symbol použít jako název funkce, uložíme do něj funkci jako hodnotu, stejně jako kdybychom ho použili jako proměnnou.

Ve Scheme se ovšem funkcím říká **procedury**, takže tento termín budeme pro funkce jazyka Scheme používat i my.

Při vyhodnocení tohoto výrazu:

```
(* 2 3)
```
Scheme

dostaneme jako výsledek `6`, ale jednodušším způsobem než v Lispu: první prvek seznamu, tedy symbol `*` se prostě **vyhodnotí** (tak to v Lispu nebylo) a pak se aplikuje procedura (v tomto případě vestavěná), která je jeho hodnotou.

Takto funguje ve Scheme vyhodnocování složených výrazů: při vyhodnocování seznamu

```
(op arg1 arg2 ... argn)
```
Scheme

se vykonají tyto kroky:

1. Jestliže *`op`* je speciální operátor, celý výraz se vyhodnotí podle jeho pravidel.
2. Jinak se *`op`* **vyhodnotí**. Výsledkem **musí být procedura**.
3. Potom se vyhodnotí argumenty a procedura se na ně aplikuje jako v Lispu.

Ve Scheme tedy operátor nemusí být symbol. Může to být libovolný výraz, který se vyhodnotí na proceduru:

```
> ((if (> 1 0) + -) 1 1)
2
```
Scheme

Nepotřebujeme tedy operátory `function` a `funcall`. Z tohoto pohledu je Scheme jednodušší než Lisp. Na rozdíl od Lispu si ale musíme dávat pozor, abychom nepojmenovávali proměnné stejně jako funkce. Symbol `*` by například nešlo používat jako proměnnou Listeneru.

3 Principy interpretu

Funkce `ss-eval` bude realizovat **interpret** jazyka Scheme; přesněji řečeno, interpret bude program skládající se z více funkcí a funkce `ss-eval` bude jeho (jediným) **vstupním bodem**.

Interpret programovacího jazyka je program, který vykonává program zadaný zdrojovým kódem napsaným v programovacím jazyce krok po kroku (výrazu, příkazu). Vykonává přímo úlohu, která je ve zdrojovém kódu zapsána, aniž by program překládal do jiného jazyka.

Čtenář se asi setkal i s pojmem **kompilátor**. To je něco zcela jiného, než interpret. *Kompilátor* (*překladač*) programovacího jazyka je program, který převádí (překládá) program zapsaný v jednom programovacím jazyce do jiného (obvykle jednoduššího, nižšího) programovacího jazyka. Sám přitom obvykle žádné instrukce

programu nevykonává. Přeložený program slouží pak jako vstup jinému interpretu případně kompilátoru.

V *PP Polyglotu* je k dispozici jak kompilátor, tak interpret jazyka. Kompilátor se spouští při vyhodnocování výrazu v editoru. Překládá program do strojového kódu procesoru, který je pak vykonán počítačem. Interpret je používán v Listeneru.

Zadání pro interpret

- vyhodnocování symbolů a ostatních atomů,
- vyhodnocování složených výrazů,
- prostředí,
- uživatelské funkce jako uzávěry, jejich aplikace,
- vestavěné funkce (*primitiva*) (jako aritmetické operace a funkce na práci se seznamy),
- čtyři základní speciální operátory.

Interpret bude napsán přesně podle zásad vyhodnocovacího procesu našeho jazyka Scheme, tedy tak, jak jsme se učili během semestru se zjednodušením uvedeným na začátku této přednášky. Uvidíme tedy, že vyhodnocovací proces je možné naprogramovat.

Principy implementace

1. Při interpretaci programu budeme využívat hodnoty Lispu (páry, symboly, čísla . . .).
2. Dále budeme využívat primitiva Lispu (např. funkci na sčítání čísel).
3. Program k interpretaci budeme zadávat jako hodnotu jazyka Lisp (atom nebo seznam), nikoliv v textové podobě, jak je obvyklé. Tím odpadne potřeba analyzovat textový zápis programu.
4. Interpret napíšeme čistě funkcionálně, pouze s použitím znalostí z tohoto semestru.

Čistě funkcionální interpret

- prostředí nelze modifikovat (neexistuje operátor `bind`)
- jak napsat rekurzi?

Použití funkce `ss-eval` (základní výrazy)

```
> (ss-eval 1)
1

> (ss-eval 'pi)
3.141592653589793D0

> (ss-eval '(quote a))
A

> (ss-eval '(+ 1 2))
3

> (ss-eval '((if (> 1 0) + -) 1 1))
2
```

Použití funkce `ss-eval` (vazby a procedury)

```
> (ss-eval '(let ((x 1)
                  (y 2))
              (+ x y)))
3

> (ss-eval '(let ((x 1)
                  (y 2))
              ((lambda (x) (* x y)) 5)))
10

> (ss-eval '(let ((avg (lambda (x y)
                         (/ (+ x y) 2))))
              (avg 1 3)))
2
```

Použití funkce `ss-eval` (rekurze)

```
> (ss-eval '(let ((fact (lambda (n)
                          (let ((fct (lambda (f n)
                                       (if (= n 0)
                                           1
                                         (* n (f f (- n 1)))))))
                            (fct fct n)))))
              (fact 5)))
120
```

Uvedená ukázka obsahuje použití všech možností našeho malého Scheme: vyhodnocování atomu (první dva výrazy) a seznamu (ostatní), použití čtyř speciálních operátorů: `quote` (ve třetím výrazu), `if`, `let`, `lambda`. Dále: použití primitiva (4 a další), vyhodnocení obecného výrazu na prvním místě seznamu a symbolu obsahujícího funkci na jiném (zejména 5), vytváření prostředí operátorem `let` (6 a další), aplikací uživatelské funkce (7 a další). Osmý příklad ukazuje, jak definovat a použít pojmenovanou uživatelskou funkci, devátý, jak definovat a použít pojmenovanou rekurzivní uživatelskou funkci.

Takto navržená funkce `ss-eval` by ovšem nebyla schopna provést vyhodnocení libovolného výrazu podle všech pravidel, která jsme si říkali; např. výrazu `(+ x y)` v šestém příkladě. Obecný vyhodnocovací proces totiž musí vyhodnocovat daný výraz v daném **prostředí**. Součástí našeho úkolu bude tedy reprezentovat prostředí vhodnou datovou strukturou a naprogramovat vyhodnocování výrazů v zadaném prostředí.

Funkce `ss-eval-in-env`

Na vyhodnocení výrazu v daném prostředí napíšeme funkci `ss-eval-in-env`, která bude mít druhý parametr, jímž budeme zadávat (vhodně reprezentované) prostředí. Pokud by například v prostředí `env` byla vazba symbolu `x` na hodnotu `1` a symbolu `y` na hodnotu `2`, pak by mělo jít udělat toto:

```
> (ss-eval-in-env '(+ x y) env)
3

> (ss-eval-in-env '((lambda (x) (* x y)) 5) env)
10
```

4 Zásady vytváření interpretu

Interpret Scheme bude zatím zdaleka nejdelší program, který jsme v tomto předmětu psali. Proto se budeme přísně držet základních zásad psaní programu, abychom se v tom neztratili. (To bude poučné i pro vaši budoucí práci.)

1. Každý složitější problém, který budeme řešit, rozdělíme na jednodušší podproblémy. Ty pak opět rozdělíme a budeme v tom postupovat tak dlouho, až dostaneme triviální problémy, které půjde snadno naprogramovat pomocí krátkých funkcí. Žádná funkce, kterou budeme psát, tedy pokud možno nebude přímo řešit více než jeden problém. Pokud by měla, raději v jejím těle jen zavoláme jiné funkce.

2. Program budeme udržovat stále funkční. Každou úpravu a každé doplnění pečlivě naplánujeme a pak je uděláme tak, aby program opět (byť omezeně) fungoval. To vždy důkladně otestujeme.

V těchto poznámkách (a přidružené prezentaci) můžete číst, jak program postupně vznikal. Vzhledem k tomu, že se to dělo postupnou úpravou různých jeho částí, je pořadí definic ve výsledném zdrojovém kódu jiné, než jaké vidíte tady.

5 Interpret

5.1 Funkce `ss-eval` a `ss-eval-in-env`

Jak jsme řekli, funkce `ss-eval` bude vstupním bodem programu. Víme, že Scheme vyhodnocuje výrazy v daném prostředí. I když zatím nevíme, jak budeme prostředí reprezentovat, je jasné, že musíme napsat funkci na vyhodnocení výrazu v daném prostředí. Bude to funkce `ss-eval-in-env`. Ji bude funkce `ss-eval` volat s prostředím rovným globálnímu prostředí. Funkce se tedy budou aplikovat takto:

```
(ss-eval expr)
(ss-eval-in-env expr env)
```

kde

expr je libovolný výraz a

env prostředí, ve kterém se má vyhodnotit.

Ještě jednou opakuji: zatím nevíme, jak budeme prostředí reprezentovat. To nám ale zatím nevadí. Určitě to bude nějakou datovou strukturou, ve které budou uloženy potřebné informace (o tom později).

Samozřejmě budeme potřebovat globální prostředí. Na jeho vytvoření si později napíšeme funkci `ss-global-environment`.

Funkci `ss-eval` tedy můžeme napsat takto:

```
(defun ss-eval (expr)
  (ss-eval-in-env expr (ss-global-environment)))
```

Teď se pustíme do funkce `ss-eval-in-env`.

První krok, který musí vyhodnocovací proces při vyhodnocování výrazu udělat, je zjistit, jestli je výraz atom, nebo složený výraz (tedy pár; prázdný seznam je atom). Podle toho pak proces postupuje dál. Věrni zásadě psát funkce co nejjednodušeji si na problémy vyhodnotit atom a vyhodnotit (neprázdný) seznam napíšeme později samostatné funkce (tedy rozdělíme složitý problém na dva jednodušší). Funkce `ss-eval-in-env` tedy bude jednoduchá:

```
(defun ss-eval-in-env (expr env)
  (if (ss-compound-expr-p expr)
      (ss-eval-compound-expr expr env)
    (ss-eval-atom expr env)))
```

Na zjištění, zda je výraz složený, si napíšeme predikát css-compound-expr-p. Je jasné, že pokud je výraz napsán správně, je složený právě když je párem:

```
(defun ss-compound-expr-p (expr)
  (consp expr))
```

Funkce popisuje přesně to, co se děje na začátku vyhodnocování výrazu v daném prostředí: zjistí se, zda jde o složený výraz, a pokud ano, vyhodnotí se výraz postupem pro složený výraz (v jiné funkci). Pokud ne, vyhodnotí se jako atom.

5.2 Vyhodnocování neproměnných atomů

Funkci `ss-eval` zatím nemůžeme otestovat, protože žádná větev funkce `ss-eval--in-env` není dosud implementována. Jednodušší bude zřejmě napsat funkci `ss--eval-atom`, tak to uděláme v tomto kroku.

Víme, že zvláštním typem atomu je symbol a vyhodnocuje se jinak než ostatní atomy, které se vyhodnocují jednoduše samy na sebe. Tuto část vyhodnocovacího procesu tedy můžeme rovnou napsat:

```
(defun ss-eval-atom (atom env)
  (if (symbolp atom)
      (ss-symbol-value atom env)
    atom))
```

(Využili jsme predikát `symbolp` Lispu, který zjišťuje, zda zadaná hodnota je symbol.)

Nyní už náš interpret něco umí: vyhodnocovat neproměnné atomy. Můžeme to vyzkoušet:

```
> (ss-eval 1)
1
```

(Pokus o vyhodnocení symbolu nebo seznamu by samozřejmě zatím vedl k chybě.)

5.3 Prostředí a vyhodnocování symbolů

Funkce `ss-symbol-value` použitá výše má vrátit hodnotu symbolu v daném prostředí. Abychom ji mohli napsat, musíme už pořádně definovat prostředí.

Prostředí reprezentujeme datovou strukturou.

Víme, že prostředí musí obsahovat následující informace:

1. tabulku vazeb symbolů,
2. svého předka.

Předkem prostředí je opět prostředí. Pokud prostředí předka nemá, vyjádříme to hodnotou `nil`.

Tabulku vazeb symbolů budeme reprezentovat dvěma stejně dlouhými seznamy: seznamem symbolů a seznamem jejich hodnot (jsou to vlastně sloupce tabulky). Celkově budeme prostředí reprezentovat seznamem následujícího tvaru:

```
(ss-env symbols values parent)
```

Je to tedy čtyřprvkový seznam. První jeho prvek je symbol `ss-env`. Sám o sobě nemá žádný význam, jen usnadňuje kontrolu toho, že zadaný seznam reprezentuje prostředí. Další dva prvky, jak už bylo řečeno, jsou seznam symbolů a hodnot. Například seznamy

```
(a b)
(0 1)
```

představují vazbu symbolu `a` na hodnotu `0` a symbolu `b` na hodnotu `1`.

Čtvrtý prvek seznamu, *`parent`*, je předek prostředí, což je buď jiné prostředí, nebo `nil` (v případě, že prostředí nemá předka).

Když jsme si vyjasnili tvar struktury reprezentující prostředí, napíšeme její konstruktor a selektory.

```
(defun ss-make-env (symbols values parent)
  (list 'ss-env symbols values parent))
```

Konstruktor `ss-make-env` vytvoří nové prostředí ze zadaného seznamu symbolů, seznamu hodnot a předka. Například následující vyhodnocení vytvoří prostředí s vazbou symbolu `a` na 0 a symbolu `b` na 1, které nemá předka:

```
> (ss-make-env '(a b) '(0 1) nil)
(SS-ENV (A B) (0 1) NIL)
```

Takto by tedy mohlo být vytvořeno prostředí, jehož předkem je jiné prostředí:

```
> (ss-make-env '(a b)
               '(0 1)
               (ss-make-env '(x y) '(u v) nil)))
(SS-ENV (A B) (0 1) (SS-ENV (X Y) (U V) NIL))
```

Selektory datové struktury prostředí budou vracet seznam symbolů, jejich hodnot a předka:

```
(defun ss-env-symbols (env)
  (second env))

(defun ss-env-values (env)
  (third env))

(defun ss-env-parent (env)
  (fourth env))
```

Funkce `ss-symbol-value`, kterou se snažíme napsat, má zjistit hodnotu vazby symbolu v daném prostředí. To obnáší několik věcí:

1. zjistit, zda má symbol vazbu v daném prostředí,
2. pokud nemá, hledat vazbu rekurzivně v předcích prostředí,
3. vrátit hodnotu nalezené vazby.

Když zatím odhlédneme od toho, jak se vazba symbolu v prostředí hledá, můžeme funkci `ss-symbol-value` napsat takto:

```
(defun ss-symbol-value (symbol env)
  (let ((index (position symbol (ss-env-symbols env))))
    (if (null index)
        (ss-symbol-value symbol (ss-env-parent env))
      (nth index (ss-env-values env)))))
```

(Funkci `position` jste programovali v úlohách k přednášce 5.)

Nyní bychom měli pečlivě otestovat jednak funkci `ss-symbol-value` a jednak celý vylepšený interpret. Znovu si vytvoříme prostředí z předchozího testu.

```
> (bind env (ss-make-env '(a b)
                          '(0 1)
                          (ss-make-env '(x y) '(u v) nil)))
(SS-ENV (A B) (0 1) (SS-ENV (X Y) (U V) NIL))


> (ss-eval-in-env 'a env)
0

> (ss-eval-in-env 'y env)
V
```

Nyní můžeme také definovat globální prostředí. Uložíme do něj několik primitiv, která budeme v budoucnu potřebovat a pro zajímavost také proměnnou `zero` obsahující nulu. Globální prostředí vytvoříme funkcí `ss-global-environment`.

```
(defun ss-global-environment ()
  (ss-make-env '( zero pi + - * / = cons)
               (list 0 pi #'+ #'- #'* #'/ #'= #'cons)
               nil))
```

Na našem testovacím prostředí `env`, které jsme definovali před chvílí, teď můžeme vyzkoušet i hodnoty proměnných uložených v globálním prostředí:

```
> (ss-symbol-value 'zero (ss-global-environment))
0
```

Také můžeme konečně použít funkci `ss-eval` a zkusit vyhodnotit výrazy v globálním prostředí:

```
> (ss-eval 'zero)
0

> (ss-eval '=)
#<FUNCTION =>
```

5.4 Vyhodnocování složeného výrazu a aplikace primitiv

Už jsme naučili náš interpret vyhodnocovat atomy. Nyní se pustíme do vyhodnocování neprázdných seznamů. Musíme tedy naprogramovat funkci `ss-eval-compound-expr`.

Víme, že při vyhodnocování seznamu se má systém nejprve podívat na první prvek (tedy operátor) a pokud půjde o speciální operátor, má zvolit příslušný speciální způsob vyhodnocení. Pokud o speciální operátor nepůjde, provede se standardní vyhodnocení, které povede na aplikaci procedury.

Případ speciálního operátoru lze řešit různými chytrými způsoby. My se budeme držet při zemi a v jednom velkém větvení postupně otestujeme, zda první prvek seznamu není jedním ze stanovených speciálních operátorů. Pokud nebude, budeme vědět, že jde o aplikaci procedury.

Pro začátek napíšeme funkci `ss-eval-compound-expr` pro základní speciální operátory: `quote`, `if`, `lambda` a `let`.

Funkce `ss-eval-compound-expr`

- výrazy s operátory `quote`, `if` a `lambda` vyhodnotí podle jejich pravidel,
- `let`-výraz převede na jiný výraz a ten vyhodnotí (jako *makro*),
- ostatní vyhodnotí jako aplikaci procedury.

```
(defun ss-eval-compound-expr (e env)
  (let ((op (car e))
        (args (cdr e)))
    (cond ((eql op 'quote)
           (apply #'ss-eval-quote-expr args))
          ((eql op 'if)
           (apply #'ss-eval-if-expr env args))
          ((eql op 'lambda)
           (apply #'ss-eval-lambda-expr env args))
          ((eql op 'let)
           (ss-eval-in-env (ss-expand-let-expr (first args)
                                               (second args))
                           env))
          (t (ss-eval-application env e)))))
```

Funkce `ss-eval-quote-expr`, `ss-eval-if-expr` a `ss-eval-lambda-expr` slouží k vyhodnocení speciálních výrazů s operátory `quote`, `if` a `lambda`. Napíšeme je později.

Výraz s operátorem `let` lze vždy přepsat na jiný výraz, který tento operátor nepoužívá. To dělá funkce `ss-expand-let-expr`, jejíž návratová hodnota se pak opět vyhodnotí funkcí `ss-eval-in-env`. Funkci `ss-expand-let-expr` si také napíšeme později.

Teď se budeme věnovat základnímu případu vyhodnocení seznamu, kdy operátor není speciálním operátorem. Jinými slovy, teď se pustíme do funkce `ss-eval-application`.

Jak víme, při vyhodnocování seznamu, jehož první prvek není speciální operátor, je nejprve třeba vyhodnotit první a potom všechny ostatní prvky seznamu. Hodnota prvního prvku musí být procedura, ta se aplikuje na hodnoty zbylých prvků. Máme tedy návod, jak napsat funkci `ss-eval-application`:

```
(defun ss-eval-application (env expr)
  (let ((vals (ss-eval-exprs expr env)))
    (ss-apply-proc (car vals) (cdr vals))))
```

Pomocná funkce:

```
(defun ss-eval-exprs (list env)
  (mapcar (lambda (expr)
            (ss-eval-in-env expr env))
          list))
```

(doplníme na začátek souboru)

Funkce `ss-apply-proc`, která je teď na řadě, má aplikovat proceduru na (již vyhodnocené) argumenty. Jak se aplikace má provést, záleží na tom, zda je procedura uživatelsky definovaná, nebo jde o primitivum. Druhá varianta je jednodušší, proto se nejprve pustíme do ní.

Jako primitiva budeme chápat procedury, které jsou definovány jako funkce Lispu. Že je hodnota funkcí v Lispu, poznáme pomocí predikátu `functionp`. Funkce `ss-apply-proc` se podívá, zda má aplikovat primitivum, nebo uživatelskou proceduru, a podle toho zavolá příslušnou pomocnou funkci:

```
(defun ss-apply-proc (proc args)
  (if (ss-primitive-p proc)
      (ss-apply-primitive proc args)
    (ss-apply-user-proc proc args)))
```

```
(defun ss-primitive-p (proc)
  (functionp proc))
```

Funkci `ss-apply-primitive` napíšeme snadno:

```
(defun ss-apply-primitive (proc args)
  (apply proc args))
```

Tím jsme náš interpret naučili aplikovat primitivní procedury:

```
> (ss-eval '(+ 1 2))
3

> (ss-eval '(cons (+ 1 2) (* 3 4)))
(3 . 12)
```

Aplikace uživatelské procedury bude později.

5.5 Speciální operátory `quote` a `if`, makro `let`

K zavedení speciálních operátorů je třeba napsat příslušné funkce, které volá funkce `ss-eval-compound-expr`. U speciálních operátorů `quote` a `if` to bude jednoduché:

```
(defun ss-eval-quote-expr (arg)
  arg)
```

Test:

```
> (ss-eval '(quote a))
A

> (ss-eval '(quote (a b c)))
(A B C)

> (ss-eval ''x)
X
```

Speciální operátor `if`:

```
(defun ss-eval-if-expr (env test then else)
  (if (ss-eval-in-env test env)
      (ss-eval-in-env then env)
    (ss-eval-in-env else env)))
```

Test:

```
> (ss-eval '(if 't 1 2))
1

> (ss-eval '(if 'nil 1 2))
2
```

```
> (ss-eval '(if (= 1 2) (+ 1 2) (* 3 4)))
12
```

Operátor `let` definujeme jako **makro**:

1. `let`-výraz převedeme na jiný výraz,
2. ten vyhodnotíme.

(druhé jsme už udělali ve funkci `ss-eval-compound-expr`)

Převod (expanze) `let`-výrazu

```
              (let ((a 1)              ((lambda (a b) (+ a b))
                    (b 2))      ->      1
                (+ a b))                2)
```

Implementujeme ve funkci:

```
(defun ss-expand-let-expr (bindings body)
  (cons (list 'lambda
              (mapcar #'first bindings)
              body)
        (mapcar #'second bindings)))
```

5.6 Uživatelské procedury

Nyní se pustíme do speciálního operátoru `lambda` a uživatelských procedur. Jak víme, uživatelské procedury obsahují tři údaje:

1. seznam parametrů,
2. tělo,
3. prostředí vzniku.

Budeme je reprezentovat datovou strukturou, která tyto údaje bude obsahovat. Struktura bude seznamem, který bude kromě nich obsahovat ještě symbol `ss-proc` kvůli snadnější orientaci:

```
(ss-proc params body env)
```

Konstruktor a selektory uživatelských procedur:

```
(defun ss-make-proc (params body env)
  (list 'ss-proc params body env))

(defun ss-proc-params (proc)
  (second proc))

(defun ss-proc-body (proc)
  (third proc))

(defun ss-proc-env (proc)
  (fourth proc))
```

Uživatel našeho malého Scheme bude procedury vytvářet speciálním operátorem `lambda`. V něm zadává seznam parametrů vytvářené procedury a její tělo. Prostředí, ve kterém je λ-výraz vyhodnocován, je také známo.

Jak jsme si stanovili ve funkci `ss-eval-compound-expr`, o vyhodnocení λ-výrazů se má starat funkce `ss-eval-lambda-expr`. Té tedy stačí zavolat konstruktor `ss-make-proc`:

```
(defun ss-eval-lambda-expr (env params body)
  (ss-make-proc params body env))
```

Poslední, co je v našem interpretu třeba udělat, je napsat funkci `ss-apply-user-proc` na aplikaci uživatelské procedury. Zopakujme si, co se má během této aplikace stát:

1. Vytvořit nové prostředí, ve kterém budou parametry procedury navázány na argumenty a jehož předkem bude prostředí vzniku procedury.
2. V tomto prostředí vyhodnotit tělo procedury.

```
(defun ss-apply-user-proc (proc args)
  (ss-eval-in-env (ss-proc-body proc)
                  (ss-make-env (ss-proc-params proc)
                               args
                               (ss-proc-env proc))))
```

Interpret je hotov a lze jej libovolně testovat.

Otázky a úkoly na cvičení

1. Příklad z přednášky na vyhodnocení výrazu

```
(ss-eval '((if (> 1 0) + -) 1 1))
```

   vede k chybě. Vysvětlete ji a upravte kód interpretu tak, aby příklad fungoval.

2. Otestujte důkladně interpret na vybraných příkladech, které jste v tomto předmětu během semestru programovali, včetně jednoduchých rekurzivních funkcí. V případě potřeby si rozšiřte globální prostředí o nové vazby na primitiva.

3. Doplňte do interpretu speciální operátor `let*`. Udělejte to pomocí už napsaného operátoru `let`.

4. Proč v testech funkce `ss-eval-if-expr` musely být symboly `t` a `nil` zakvotované? Vysvětlete a opravte.

5. Definujte v našem Scheme (samozřejmě lokální) funkce odpovídající funkcím `even-conses` a `odd-conses` z přednášky 6.

6. Funkce Lispu `symbol-function` vrací k symbolu funkci, jíž je jménem v globálním prostředí:

```
> (symbol-function 'list)
#<FUNCTION LIST>
```

   (Funkce tedy nefunguje na lokální funkce definované operátorem `labels`.)

   Rozšiřte náš interpret o speciální operátor `primitive`, který vrátí funkci Lispu (z pohledu Scheme tedy primitivum) daného názvu:

```
> (ss-eval '(primitive list))
#<FUNCTION LIST>

> (ss-eval '((primitive list) 1 2 3))
(1 2 3)
```